Putting it together

The series on one map, choosing a list, tuple, set or dict, and one problem built step by step.

THE QUESTION

You know the pieces. How do you turn a problem into a program that works, and know it does?

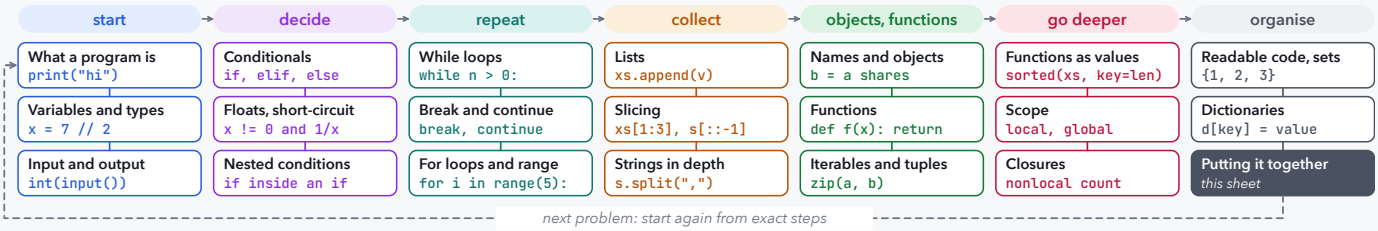
THE INTUITION

Pick a structure for the question you will ask. Build small functions and run each on a tiny input. When an answer is wrong, print the data between steps to find the step.

KEY TAKEAWAY

One small function per step, run and tested with `assert`. When it breaks, read the traceback from the bottom.

1 Twenty sheets in seven stages, then one problem that uses them all



Each box is a sheet and a line of the code it taught. Below, one problem uses them together: read, clean, convert, compute, collect in a dict, sort, print with an f-string.

2 Pick a structure for your question

	list	tuple	set	dict
use it for	items in order	a fixed record	is v in? once each	look up by key
example, run	`sorted([8, 7])` → `[7, 8]`	`parse("Bob,8")` → `('Bob', 8)`	`{8, 7, 8}` → `{8, 7}`	`scores["Bob"]` → `[6, 10]`
keeps order	yes	yes	no	yes, as added
can change	yes	no	yes	yes
repeats	yes	yes	no	keys: no
fast at	`xs[i]`, `append`	`t[i]`	`v in s`, `add`	`d[k]`, `k in d`
slow at	`v in xs`	`v in t`	no `s[i]`	`v in d.values()`
`-1 in x` 100,000 items	442 µs	427 µs	8 ns	9 ns

time for one lookup of a missing value: best of 5 runs on this machine; yours will differ

A **data structure** holds many values under one name. `v in xs` on a list checks item after item, so it slows as the list grows; a set or dict jumps to one slot by the value's **hash** (the sets sheet), here about 54,000 times faster. Structures nest: `scores` is a dict of lists. The last notebook builds a letter table with `dict(zip(abc, abc[2:] + abc[:2]))`, turning `map` into `ocr`, the first sheet's puzzle.

3 Steps 1 to 3: read, clean, convert

steps 2 and 3: clean, then convert

```
10 def parse(line):
11     name, score = line.split(",")
12     name = name.strip().title()
13     return name, int(score)
```

step 1: read_lines, lines 1 to 7

a sentinel loop: append each typed line until one says end

line 11: cut at the comma
line 12: strip spaces, Capitalise
line 13: int() makes the number

typed line	line.split(",")	name, cleaned	parse(line)
`'Alice, 8'`	`['Alice', ' 8']`	`'Alice'`	`('Alice', 8)`
`'bob,6'`	`['bob', '6']`	`'Bob'`	`('Bob', 6)`
`'  Carol , 9'`	`['  Carol ', ' 9']`	`'Carol'`	`('Carol', 9)`
`'alice,7'`	`['alice', '7']`	`'Alice'`	`('Alice', 7)`
`''`	*blank: collect skips it before parse (step 5)*		
`'Bob, 10'`	`['Bob', ' 10']`	`'Bob'`	`('Bob', 10)`

The task: people type `name, score` lines with stray spaces, mixed capitals and a blank line, then `end`; print each name's count and average, best first. Write one small function per step and run it on this input before writing the next. `parse` returns a tuple.

4 Steps 4 and 5: compute, collect in a dict

step 5: collect in a dict

```
20 def collect(lines):
21     scores = {}
22     for line in lines:
23         if line.strip() == "":
24             continue
25         name, score = parse(line)
26         if name not in scores:
27             scores[name] = []
28         scores[name].append(score)
29     return scores
```

step 4: compute with a function

```
16 def average(xs):
17     return sum(xs) / len(xs)
```

```
average([6, 10])
→ 8.0
```

grey: skip a blank line
amber: a new name gets []
scores: a dict of lists

after line	scores
`'Alice, 8'`	`{'Alice': [8]}`
`'bob,6'`	`{'Alice': [8], 'Bob': [6]}`
`'  Carol , 9'`	`{'Alice': [8], 'Bob': [6], 'Carol': [9]}`
`'alice,7'`	`{'Alice': [8, 7], 'Bob': [6], 'Carol': [9]}`
`''`	*(unchanged)*
`'Bob, 10'`	`{'Alice': [8, 7], 'Bob': [6, 10], 'Carol': [9]}`

`average` does one computation, reused in step 6. `collect` skips blank lines, gives a new name an empty list, then appends the score to that name's list. The table shows the dict after each typed line: `bob` joins `Bob` because `parse` cleaned it.

5 Steps 6 and 7: sort by average, then print

steps 6 and 7: sort by average, then print a table

```
32 def report(scores):
33     names = sorted(scores, key=lambda n: average(scores[n]),
34                    reverse=True)
35     print(f"{'name':<6}{'n':>3}{'avg':>7}")
36     for name in names:
37         xs = scores[name]
38         print(f"{name:<6}{len(xs):>3}{average(xs):>7.2f}")
41 report(collect(read_lines()))                    line 41: every step in one line
```

the key gives each name its average

```
'Alice'  → 7.5
'Bob'    → 8.0
'Carol'  → 9.0
```

sorted, reverse=True:

```
['Carol', 'Bob', 'Alice']
```

```
$ python3 report.py
name     n    avg
Carol    1   9.00
Bob      2   8.00
Alice    2   7.50
```

typed: the six lines, then end

`sorted(scores, ...)` walks the keys, the names. The **key function** maps each name to its average; `reverse=True` puts the largest first. The **format spec** `<6` pads a name to 6 characters, `>3` and `>7.2f` right-align numbers (the input and output sheet).

When it breaks, find the step, then fix it

A debugging checklist used on a real bug, a traceback three calls deep, tests with `assert`, and what to learn next.

6 A debugging checklist, used on a real bug

```
name    n    avg
Bob     1  10.00
  Carol   1   9.00
Alice   1   8.00
alice   1   7.00
bob     1   6.00
```

report.py without line 12
(no strip, no title), fed the same six typed lines:

5 rows for 3 people, and one row pushed out of line.
No error, so no traceback to read.

1. Read the last line of the error — *none: the output is wrong instead*
2. Print the values between steps — `print(collect(lines))`
   `{'Alice': [8], 'bob': [6], '  Carol ': [9], 'alice': [7], 'Bob': [10]}`
3. Find the smallest failing input — `collect(["Bob, 10", "bob,6"])`
   → `{'Bob': [10], 'bob': [6]}`
4. Check types and exact values — `"bob" == "Bob"` → `False`
5. Explain it aloud (rubber duck) — *bob and Bob are two keys*

fix: put line 12 back, `name = name.strip().title()`, and rerun: 3 rows

A **bug** makes a program crash or answer wrongly. This one gives no error, only five names for three people. Explaining code aloud, even to a toy duck, is **rubber-duck debugging**.

7 Read a nested traceback bottom up

```
Traceback (most recent call last):
  File ".../report.py", line 39, in <module>
    report(collect(read_lines()))
           ^^^^^^^^^^^^^^^^^^^^^
  File ".../report.py", line 23, in collect
    name, score = parse(line)
                  ^^^^^^^^^^^
  File ".../report.py", line 11, in parse
    name, score = line.split(",")
    ^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

1. What went wrong: unpacking wanted 2 values, got 1.
2. Where: parse, line 11, the deepest call. Look here first.
3. Who called it: collect, line 23, handing it one line.
4. Who called that: line 39, the first call, at the top.

print(repr(line)) above line 23 shows '' last: the blank line.

fix: skip blank lines, as lines 23 and 24 of report.py do

Without the blank-line check, the blank line reaches `parse`. A **traceback** has one **frame** per unfinished call. Read the last line, then the frame above; go higher only to see who passed the bad value.

8 Test every step with `assert`

below the functions, in place of line 41 (or in the next notebook cell)

```
41 assert parse("Alice, 8") == ("Alice", 8)
42 got = parse("  bob ,10")
43 assert got == ("Bob", 10), f"got {got}"
44 assert average([6, 10]) == 8.0
45 assert collect(["a,1", "", "A,3"]) == {"A": [1, 3]}
46 assert collect([]) == {}
47 print("all tests passed")
```

as written, it prints `all tests passed`

without line 12 (the bug in the checklist), line 43 stops it with

```
AssertionError: got '  bob ', 10)
```

`assert test, message` stops with an **AssertionError** when the test is False (the functions sheet). Test each function on small cases, awkward ones too: spaces, capitals, a blank line, no lines. Rerun all tests after every fix.

9 Where to go next

Files
```
with open("scores.txt") as f:
    lines = f.read().splitlines()
lines[:2] → ['Alice, 8', 'bob,6']
```

Modules
```
import statistics
statistics.mean([6, 10])
→ 8
```

Classes: your own types
```
s = Student("Bob", [6, 10])
s.average()
→ 8.0
```

Libraries: NumPy
```
import numpy as np
np.array([6, 10]).mean()
→ 8.0
```

A **module** is a `.py` file you `import`, yours too. A **class** is your own type: data plus its functions. **NumPy** works on whole arrays; the last notebook checks sudoku rows with it. `dir(x)` lists what x can do.

10 Common mistakes when joining the steps: what you wrote, what Python says, the fix

```
sum(["8", "7"])
TypeError: unsupported operand
type(s) for +: 'int' and 'str'
```
text: `int()` each first; sum 15

```
scores["Alice"].append(8)
KeyError: 'Alice'
```
new name, no list:
`scores[name] = []`

```
name, n = "Al,8,9".split(",")
ValueError: too many values to
unpack (expected 2)
```
check it, or `split(",", 1)`

```
average([])
ZeroDivisionError: division by zero
```
no scores: `if not xs: return 0.0`

```
sorted(scores, key=average)
TypeError: unsupported operand
type(s) for +: 'int' and 'str'
```
key gets names:
`lambda n: average(scores[n])`

```
collect(read_lines)
TypeError: 'function' object is not
iterable
```
pass its result:
`collect(read_lines())`

11 Try it (answers in green)

1. Count the different words of s: which structure?
   A set: `len(set(s.split()))`

2. `"Bob, 7".split(",")`?
   `['Bob', ' 7']`; `int` of piece 1 is 7

3. `f"{'Bob':<5}|{8.0:>6.2f}|"`?
   `Bob  |   8.00|`

4. Frames `<module>` 12, `f` 8, `g` 2, then ZeroDivisionError: where?
   Line 2, in `g`: the bottom frame.

5. `price = 1,200`, then `print(price * 2)`?
   `(1, 200, 1, 200)`: a tuple.

6. Does `assert average([2, 3]) == 2` pass?
   No, `AssertionError`: it is `2.5`.

a structure per question · a function per step · print between steps · tracebacks bottom up · assert, rerun

Source: the two conclusion notebooks (a letter table from dict and zip, dir and help, NumPy on a sudoku) and the very basics review notebooks, nipunbatra.github.io/comp22; the score report, its bugs and the problems are our own. Outputs and errors come from Python 3.11.11, typed input fed in. This is the last sheet; for your next problem, start again with the what a program is sheet.